

ALF Basics: *Observables*

Francesco Parisen Toldin

ALF Workshop 2022
15 February 2022

- ① Basics: observables type in ALF
- ② Predefined observables
- ③ Advanced: user-defined observables

- ① Basics: observables type in ALF
- ② Predefined observables
- ③ Advanced: user-defined observables

Observables and sign reweighting

- Reweighting: phase of a QMC configuration C is

$$\text{phase} = \frac{e^{-S(C)}}{|e^{-S(C)}|}$$

- Reweighting of an observable O

$$\langle \hat{O} \rangle = \frac{\left\langle \text{sgn} \frac{e^{-S}}{\text{Re}[e^{-S(C)}]} \langle \hat{O} \rangle_C \right\rangle_{\bar{P}}}{\langle \text{sgn} \rangle_{\bar{P}}}$$

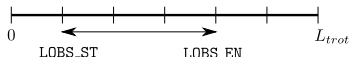
- Observables are automatically multiplied by $\text{ZS} * \text{ZP}$

$$\text{ZS} = \text{sgn}(C) \quad \text{ZP} = \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]}$$

Scalar observables

- Example: energy, magnetization, ...
- Time-average on [LOBS_ST, LOBS_EN]

```
&VAR_QMC                !! Variables for the QMC run
Nwrap                    = 10  ! Stabilization. Green functions will be computed from
                             ! scratch after each time interval Nwrap*Dtau
NSweep                   = 20  ! Number of sweeps
NBin                     = 5   ! Number of bins
Ltau                     = 1   ! 1 to calculate time-displaced Green functions; 0 otherwise
LOBS_ST                  = 0   ! Start measurements at time slice LOBS_ST
LOBS_EN                  = 0   ! End measurements at time slice LOBS_EN
CPU_MAX                  = 0.0 ! Code stops after CPU_MAX hours, if 0 or not
                             ! specified, the code stops after Nbin bins
Propose_S0               = .F. ! Proposes single spin flip moves with probability exp(-S0)
Global_moves             = .F. ! Allows for global moves in space and time
N_Global                 = 1   ! Number of global moves per sweep
Global_tau_moves         = .F. ! Allows for global moves on a single time slice.
N_Global_tau             = 1   ! Number of global moves that will be carried out on a
                             ! single time slice
```



Scalar observables

- Example: energy, magnetization, ...

- Stored into the Fortran type `Obser_vec`

Variable	Type	Description	Contribution
<code>N</code>	<code>int</code>	Number of measurements	+1
<code>Ave_sign</code>	<code>dble</code>	Cumulated average sign [Eq. (28)]	$\text{sgn}(C)$
<code>Obs_vec(:)</code>	<code>cmplx</code>	Cumulated vector of observables [Eq. (27)]	$\langle\langle\hat{O}(\cdot)\rangle\rangle_C \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]} \text{sgn}(C)$
<code>File_Vec</code>	<code>char</code>	Name of output file	
<code>analysis_mode</code>	<code>char</code>	How to analyze the observable Default value: "identity"	
<code>description(:)</code>	<code>char</code>	Optional description. Arbitrary number of 64-character lines	

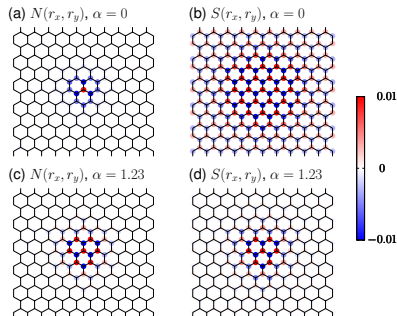
- Output: `File_Vec_scal`
- Info file: `File_Vec_scal_info`

Equal-time correlation functions

- Example: charge correlations, spin correlations, Green's function,...
- Observables of the form

$$S_{\alpha,\beta}(\vec{k}) = \frac{1}{N_{\text{unit-cell}}} \sum_{\vec{i},\vec{j}} e^{-i\vec{k}\cdot(\vec{i}-\vec{j})} \left(\langle \hat{O}_{\vec{i},\alpha} \hat{O}_{\vec{j},\beta} \rangle - \langle \hat{O}_{\vec{i},\alpha} \rangle \langle \hat{O}_{\vec{j},\beta} \rangle \right)$$

- Automatic Fourier transform



Hohenadler, Parisen Toldin, Herbut, Assaad (2014)

Time-displaced correlation functions

- Example: charge correlations, spin correlations, Green's function,...
- Observables of the form

$$S_{\alpha,\beta}(\vec{k}, \tau) = \frac{1}{N_{\text{unit-cell}}} \sum_{\vec{i}, \vec{j}} e^{-i\vec{k} \cdot (\vec{i} - \vec{j})} \left(\langle \hat{O}_{\vec{i},\alpha}(\tau) \hat{O}_{\vec{j},\beta} \rangle - \langle \hat{O}_{\vec{i},\alpha} \rangle \langle \hat{O}_{\vec{j},\beta} \rangle \right)$$

- **Expensive:** activate with Ltau=1

&VAR_QMC		!! Variables for the QMC run
Nwrap	= 10	! Stabilization. Green functions will be computed from scratch after each time interval Nwrap*Dtau
NSweep	= 20	! Number of sweeps
NBin	= 5	! Number of bins
Ltau	= 1	! 1 to calculate time-displaced Green functions; 0 otherwise
LOBS_ST	= 0	! Start measurements at time slice LOBS_ST
LOBS_EN	= 0	! End measurements at time slice LOBS_EN
CPU_MAX	= 0.0	! Code stops after CPU_MAX hours, if 0 or not specified, the code stops after Nbin bins
Propose_S0	= .F.	! Proposes single spin flip moves with probability exp(-S0)
Global_moves	= .F.	! Allows for global moves in space and time
N_Global	= 1	! Number of global moves per sweep
Global_tau_moves	= .F.	! Allows for global moves on a single time slice.
N_Global_tau	= 1	! Number of global moves that will be carried out on a single time slice

Equal-time and time-displaced correlation functions

- Example: charge correlations, spin correlations, Green's function,...
- Stored into the Fortran type `Obser_latt`

Variable	Type	Description	Contribution
<code>Obs%N</code>	<code>int</code>	Number of measurements	+1
<code>Obs%Ave_sign</code>	<code>dble</code>	Cumulated sign [Eq. (28)]	$\text{sgn}(C)$
<code>Obs%Obs_latt(i-j, τ, α, β)</code>	<code>cmplx</code>	Cumulated correlation function [Eq. (27)]	$\langle \langle \hat{O}_{i,\alpha}(\tau) \hat{O}_{j,\beta} \rangle \rangle_C \times \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]} \text{sgn}(C)$
<code>Obs%Obs_latt0(α)</code>	<code>cmplx</code>	Cumulated expected value [Eq. (27)]	$\langle \langle \hat{O}_{i,\alpha} \rangle \rangle_C \times \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]} \text{sgn}(C)$
<code>Obs%File_Latt</code>	<code>char</code>	Name of output file	
<code>Obs%Latt</code>	<code>Lattice*</code>	Bravais lattice [Tab. 4]	
<code>Obs%Latt_unit</code>	<code>Unit_cell*</code>	Unit cell [Tab. 5]	
<code>Obs%dtau</code>	<code>dble</code>	Imaginary time step	
<code>Obs%Channel</code>	<code>char</code>	Channel for Maximum Entropy	

- Output: `File_Latt_eq` (equal-time), `File_Latt_tau` (time-displaced)
- Info file: `File_Latt_eq_info` (equal-time),
`File_Latt_tau_info` (time-displaced)

- ① Basics: observables type in ALF
- ② Predefined observables
- ③ Advanced: user-defined observables

Predefined observables

- ALF provides an ever-growing list of predefined observables
- General arguments taken by `Predefined_Obs_*` routines

Argument	Type	Description
<code>Latt</code>	<code>Lattice</code>	Lattice as a variable of type <code>Lattice</code> , see Sec. 5.3
<code>Latt_Unit</code>	<code>Unit_cell</code>	Unit cell as a variable of type <code>Unit_cell</code> , see Sec. 5.3
<code>List(Ndim,2)</code>	<code>int</code>	For every site index I , stores the corresponding lattice position, <code>List(I,1)</code> , and the (local) orbital index, <code>List(I,2)</code>
<code>NT</code>	<code>int</code>	Imaginary time τ
<code>GR(Ndim,Ndim,N_FL)</code>	<code>cmplx</code>	Equal-time Green function $\text{GR}(i,j,s) = \langle c_{i,s} c_{j,s}^\dagger \rangle$
<code>GRC(Ndim,Ndim,N_FL)</code>	<code>cmplx</code>	$\text{GRC}(i,j,s) = \langle c_{i,s}^\dagger c_{j,s} \rangle = \delta_{i,j} - \text{GR}(j,i,s)$
<code>GTO(Ndim,Ndim,N_FL)</code>	<code>cmplx</code>	Time-displaced Green function $\langle\langle \mathcal{T} \hat{c}_{i,s}(\tau) \hat{c}_{j,s}^\dagger(0) \rangle\rangle$
<code>GOT(Ndim,Ndim,N_FL)</code>	<code>cmplx</code>	Time-displaced Green function $\langle\langle \mathcal{T} \hat{c}_{i,s}(0) \hat{c}_{j,s}^\dagger(\tau) \rangle\rangle$
<code>GOO(Ndim,Ndim,N_FL)</code>	<code>cmplx</code>	Time-displaced Green function $\langle\langle \mathcal{T} \hat{c}_{i,s}(0) \hat{c}_{j,s}^\dagger(0) \rangle\rangle$
<code>GTT(Ndim,Ndim,N_FL)</code>	<code>cmplx</code>	Time-displaced Green function $\langle\langle \mathcal{T} \hat{c}_{i,s}(\tau) \hat{c}_{j,s}^\dagger(\tau) \rangle\rangle$
<code>N_SUN</code>	<code>int</code>	Number of fermion colors N_{col}
<code>ZS</code>	<code>cmplx</code>	$\text{ZS} = \text{sgn}(C)$, see Sec. 5.4
<code>ZP</code>	<code>cmplx</code>	$\text{ZP} = e^{-S(C)} / \text{Re} [e^{-S(C)}]$, see Sec. 5.4
<code>Obs</code>	<code>Obser_Latt</code>	Output: one or more measurement result

Equal-time spin-spin correlations: SU(N)

```
Call Predefined_Obs_eq_SpinSUN_measure(Latt, Latt_unit, List, GR, GRC,  
                                         N_SUN, ZS, ZP, Obs)
```

For $N_{\text{FL}} = 1$

$$\text{Obs}(\mathbf{i}-\mathbf{j}, n_i, n_j) = \frac{2N}{N^2 - 1} \sum_{a=1}^{N^2-1} \langle \hat{\mathbf{c}}_{\vec{i}, n_i}^\dagger T^a \hat{\mathbf{c}}_{\vec{i}, n_i} \hat{\mathbf{c}}_{\vec{j}, n_j}^\dagger T^a \hat{\mathbf{c}}_{\vec{j}, n_j} \rangle_C \sim \sum_a \langle S_{\vec{i}, n_i}^a S_{\vec{j}, n_j}^a \rangle_C$$

- $\vec{i}, \vec{j}$: indexes on the Bravais lattice
- n_i, n_j : orbital indexes in the unit cell
- T^a : SU(N) generators

Also for $N_{\text{FL}} > 1$, see doc

Equal-time spin-spin correlations

```
Call Predefined_Obs_eq_SpinMz_measure(Latt, Latt_unit, List, GR, GRC,  
                                         N_SUN, ZS, ZP, ObsZ, ObsXY, ObsXYZ)
```

For $N_{\text{SUN}} = 1$ and $N_{\text{FL}} = 2$

$$\begin{aligned}\text{ObsZ}(\mathbf{i} - \mathbf{j}, n_i, n_j) &= 4 \langle \hat{\mathbf{c}}_{i,n_i}^\dagger S^z \hat{\mathbf{c}}_{i,n_i} \hat{\mathbf{c}}_{j,n_j}^\dagger S^z \hat{\mathbf{c}}_{j,n_j} \rangle_C \\ &\quad - 4 \langle \hat{\mathbf{c}}_{i,n_i}^\dagger S^z \hat{\mathbf{c}}_{i,n_i} \rangle_C \langle \hat{\mathbf{c}}_{j,n_j}^\dagger S^z \hat{\mathbf{c}}_{j,n_j} \rangle_C\end{aligned}$$

$$\begin{aligned}\text{ObsXY}(\mathbf{i} - \mathbf{j}, n_i, n_j) &= 2 \left(\langle \hat{\mathbf{c}}_{i,n_i}^\dagger S^x \hat{\mathbf{c}}_{i,n_i} \hat{\mathbf{c}}_{j,n_j}^\dagger S^x \hat{\mathbf{c}}_{j,n_j} \rangle_C \right. \\ &\quad \left. + \langle \hat{\mathbf{c}}_{i,n_i}^\dagger S^y \hat{\mathbf{c}}_{i,n_i} \hat{\mathbf{c}}_{j,n_j}^\dagger S^y \hat{\mathbf{c}}_{j,n_j} \rangle_C \right)\end{aligned}$$

$$\text{ObsXYZ} = \frac{2 \cdot \text{ObsXY} + \text{ObsZ}}{3}$$

Time-displaced SU(N) spin-spin correlations

```
Call Predefined_Obs_tau_SpinSUN_measure(Latt, Latt_unit, List, NT, GT0, GOT,  
                                          G00, GTT, N_SUN, ZS, ZP, Obs)
```

$$\text{Obs}(\mathbf{i} - \mathbf{j}, \tau, n_i, n_j) = \frac{2N}{N^2 - 1} \sum_{a=1}^{N^2-1} \langle \hat{\mathbf{c}}_{i,n_i}^\dagger(\tau) T^a \hat{\mathbf{c}}_{i,n_i}(\tau) \hat{\mathbf{c}}_{j,n_j}^\dagger T^a \hat{\mathbf{c}}_{j,n_j} \rangle_C$$

Time-displaced spin correlations

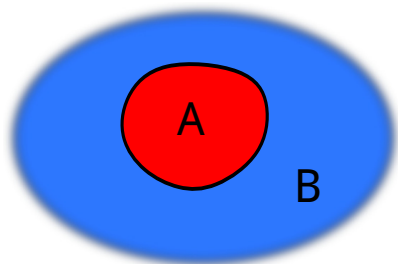
```
Call Predefined_Obs_tau_SpinMz_measure(Latt, Latt_unit, List, NT, GT0, GOT, G00,  
                                         GTT, N_SUN, ZS, ZP, ObsZ, ObsXY, ObsXYZ)
```

$$\begin{aligned}\text{ObsZ}(\mathbf{i} - \mathbf{j}, \tau, n_i, n_j) &= 4 \langle \hat{\mathbf{c}}_{i,n_i}^\dagger(\tau) S^z \hat{\mathbf{c}}_{i,n_i}(\tau) \hat{\mathbf{c}}_{j,n_j}^\dagger S^z \hat{\mathbf{c}}_{j,n_j} \rangle_C \\ &\quad - 4 \langle \hat{\mathbf{c}}_{i,n_i}^\dagger S^z \hat{\mathbf{c}}_{i,n_i} \rangle_C \langle \hat{\mathbf{c}}_{j,n_j}^\dagger S^z \hat{\mathbf{c}}_{j,n_j} \rangle_C \\ \text{ObsXY}(\mathbf{i} - \mathbf{j}, \tau, n_i, n_j) &= 2 \left(\langle \hat{\mathbf{c}}_{i,n_i}^\dagger(\tau) S^x \hat{\mathbf{c}}_{i,n_i}(\tau) \hat{\mathbf{c}}_{j,n_j}^\dagger S^x \hat{\mathbf{c}}_{j,n_j} \rangle_C \right. \\ &\quad \left. + \langle \hat{\mathbf{c}}_{i,n_i}^\dagger(\tau) S^y \hat{\mathbf{c}}_{i,n_i}(\tau) \hat{\mathbf{c}}_{j,n_j}^\dagger S^y \hat{\mathbf{c}}_{j,n_j} \rangle_C \right) \\ \text{ObsXYZ} &= \frac{2 \cdot \text{ObsXY} + \text{ObsZ}}{3}.\end{aligned}$$

Many other observables: Green's functions, Density correlations, Dimer correlations, Kondo cotunneling, ...

- Bipartite Entanglement

$$\hat{\rho}_A = \text{Tr}_B \hat{\rho}$$



- Rényi Entropy: $S_n \equiv -\frac{1}{n-1} \log \text{Tr} [\hat{\rho}_A^n]$
- Area law: [Review: J. Eisert, M. Cramer, M. B. Plenio, 2010](#)

$$S_n = a_n L^{d-1} + \dots$$

- Corrections to area law: important information on the ground-state.
E.g.: central charge in 1+1, # Goldstone modes, ...

Review: [N. Laflorencie, Phys. Rep. 646, 1 \(2016\)](#)

Second Rényi Entropy New in ALF 2

Module `entanglement_mod.F90`, requires MPI with # tasks ≥ 2

`Call Predefined_Obs_scal_Renyi_Ent(GRC, List, Nsites, N_SUN, ZS, ZP, Obs)`

returns `Obs`, such that $\langle \text{Obs} \rangle = e^{-S_2}$

- General case

`List(:, N_FL, N_SUN)` list of sites for every flavor and color
`Nsites(N_FL, N_SUN)` # lattice sites for every flavor and color
omit `N_SUN` in the call

- Common color index for every flavor

`List(:, N_FL)` list of lattice sites for every flavor
`Nsites(N_FL)` # sites for any given flavor index
`N_SUN(N_FL)` # color indexes for a given flavor index.

- Same color and flavor for every lattice site

`List(:)` list of lattice sites
`Nsites` # sites
`N_SUN` # color indexes

Mutual information between subsystems A and B

$$I_2 = S_2(A) + S_2(B) - S_2(A \cup B)$$

```
Call Predefined_Obs_scal_Mutual_Inf(GRC, List_A, Nsites_A, List_B, Nsites_B,  
                                     N_SUN, ZS, ZP, Obs)
```

- List and Nsites_A, Nsites_B as in
Predefined_Obs_scal_Renyi_Ent
- N_SUN same for A and B

- ① Basics: observables type in ALF
- ② Predefined observables
- ③ Advanced: user-defined observables

Relevant sections in Hamiltonian submodules

Prog/Hamiltonians

Observables are defined in Hamiltonian submodules, e.g.
`Prog/Hamiltonians/Hamiltonian_Hubbard_Plain_Vanilla_smod.F90`

- Subroutine `Alloc_obs(Ltau)`
Specify the equal time and time displaced observables
- Subroutine `Obser(GR,Phase,Ntau,Mc_step_weight)`
Equal-time observables
- Subroutine `ObserT(NT,GTO,GOT,G00,GTT,PHASE,Mc_step_weight)`
Time-displaced observables

User-defined observables

- Wick's theorem applies to a given QMC configuration C
- Generic observable is computed from the single-particle Green's function

$$\langle \hat{c}_{i,\sigma}^\dagger \hat{c}_{j,\sigma} \hat{c}_{k,\sigma'}^\dagger \hat{c}_{l,\sigma'} \rangle_C = \langle \hat{c}_{i,\sigma}^\dagger \hat{c}_{j,\sigma} \rangle_C \langle \hat{c}_{k,\sigma'}^\dagger \hat{c}_{l,\sigma'} \rangle_C + \langle \hat{c}_{i,\sigma}^\dagger \hat{c}_{l,\sigma'} \rangle \langle \hat{c}_{j,\sigma} \hat{c}_{k,\sigma'}^\dagger \rangle_C$$

- Only imaginary times τ and $\tau = 0$ can occur

<code>GR(Ndim,Ndim,N_FL)</code>	<code>cmplx</code>	Equal-time Green function $\text{GR}(i,j,s) = \langle c_{i,s} c_{j,s}^\dagger \rangle$
<code>GRC(Ndim,Ndim,N_FL)</code>	<code>cmplx</code>	$\text{GRC}(i,j,s) = \langle c_{i,s}^\dagger c_{j,s} \rangle = \delta_{i,j} - \text{GR}(j,i,s)$
<code>GTO(Ndim,Ndim,N_FL)</code>	<code>cmplx</code>	Time-displaced Green function $\langle \langle \mathcal{T} \hat{c}_{i,s}(\tau) \hat{c}_{j,s}^\dagger(0) \rangle \rangle$
<code>GOT(Ndim,Ndim,N_FL)</code>	<code>cmplx</code>	Time-displaced Green function $\langle \langle \mathcal{T} \hat{c}_{i,s}(0) \hat{c}_{j,s}^\dagger(\tau) \rangle \rangle$
<code>GOO(Ndim,Ndim,N_FL)</code>	<code>cmplx</code>	Time-displaced Green function $\langle \langle \mathcal{T} \hat{c}_{i,s}(0) \hat{c}_{j,s}^\dagger(0) \rangle \rangle$
<code>GTT(Ndim,Ndim,N_FL)</code>	<code>cmplx</code>	Time-displaced Green function $\langle \langle \mathcal{T} \hat{c}_{i,s}(\tau) \hat{c}_{j,s}^\dagger(\tau) \rangle \rangle$

- Missing something?

Contribute by opening an issue on GitLab or even implement it and share your solution on GitLab!